The CPU: A very tall pile of simple

Part 2: Machinery

Ekansh Goenka

October 2026

Contents

2.1. Storing a Bit 3
2.2. Buses 10
2.3. Organizing Data 15
2.4. The ALU 23
2.5. The Big Picture 26

2.1. Storing a Bit

In the last section, wiring the output of a circuit back into its input led to chaos. But with a smarter configuration, we can use this trick to store a bit.

The circuit that stores a single bit is called an SR latch. SR stands for set-reset. The value `Q` is the output we really care about. If it is `1`, that means the latch is storing a `1`; if it is `0`, the latch is storing a `0`.

The diagram also shows a second output written as a Q with a bar over it. That is just how engineers write `NOT Q`, pronounced "not Q". It always holds the opposite of `Q`. I will write it as `NOT Q` in the text.

The two inputs are `SET` and `RESET`, drawn as little buttons in the diagram: gray means not pressed, red means pressed. Pressing `SET` forces `Q` to `1` and pressing `RESET` forces `Q` to `0`.

For this circuit to be used properly, `SET` and `RESET` should never be on at the same time.

The cool part is, if both `SET` and `RESET` are `0`, then `Q` is whatever we last did to it! The output loops back into the circuit, so the current state keeps reinforcing itself. This is the basic concept behind memory.

This diagram should help this make sense:

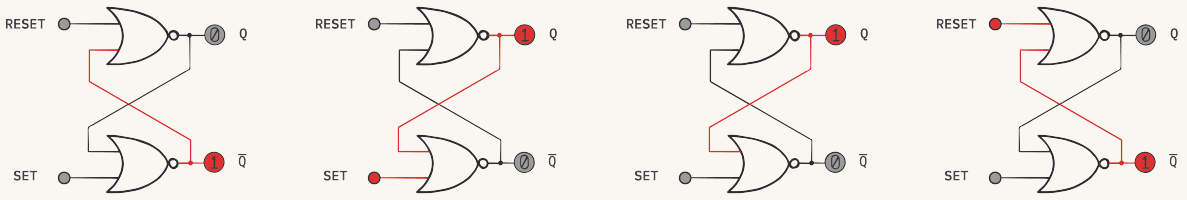


Diagram 7.1. An SR latch.

One quick reminder before we trace through this: a NOR gate only turns on when both of its inputs are off.

A simple way to think about this is:

If `SET` is on, the bottom NOR gate has to output `0`, because one of its inputs is on. That makes `NOT Q` equal to `0`.

Now the top NOR gate sees two `0` inputs: `RESET` is `0`, and `NOT Q` is `0`. So the top NOR gate outputs `1`, making `Q` equal to `1`.

Then even if we turn `SET` back off, the latch stays in that state. `Q` is still `1`, which keeps forcing `NOT Q` to `0`, and `NOT Q` being `0` allows `Q` to stay `1`.

`RESET` works the other way. If `RESET` is on, it forces `Q` to `0`, which allows `NOT Q` to become `1`. Then even after `RESET` turns off, `NOT Q` keeps forcing `Q` to stay `0`.

The circuit has state. Its output depends not only on the current input, but on what happened before.

Now that we have the core mechanism, let's refine the interface. Right now `SET` and `RESET` are super clunky. While they demonstrate the mechanism, what we would really like to have is two inputs.

- `Data` (`D`)
- `Enable` (`E`)

When the enable wire turns on, `Data` gets stored in `Q`. Or in other words, when we turn the enable wire on, `Q` mirrors `D`. Then when we turn `E` off, `Q` stays stable with whatever `D` was last.

This type of latch is called a D latch, D meaning data. It can be made using the SR latch and a few extra logic gates.

It basically checks: if `D` is true and `E` is true, `SET` is true, and if `D` is false and `E` is true, `RESET` is true. That's it, so let's not worry about the exact implementation.

If you really want to know how it works, have a look at this site.

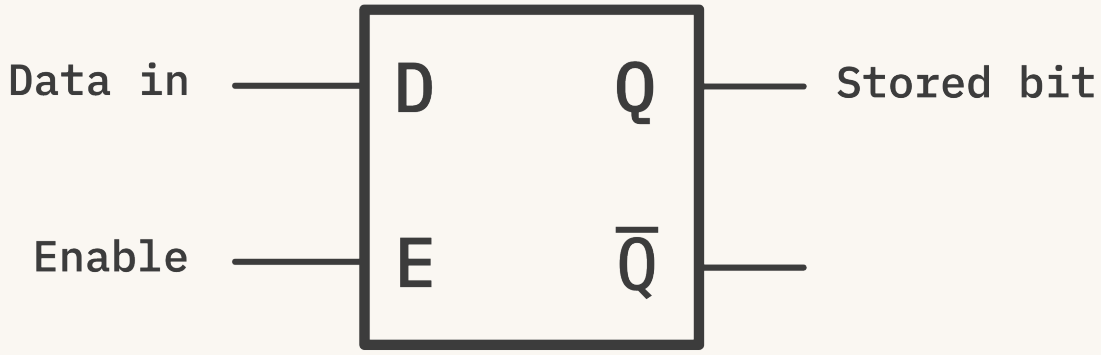
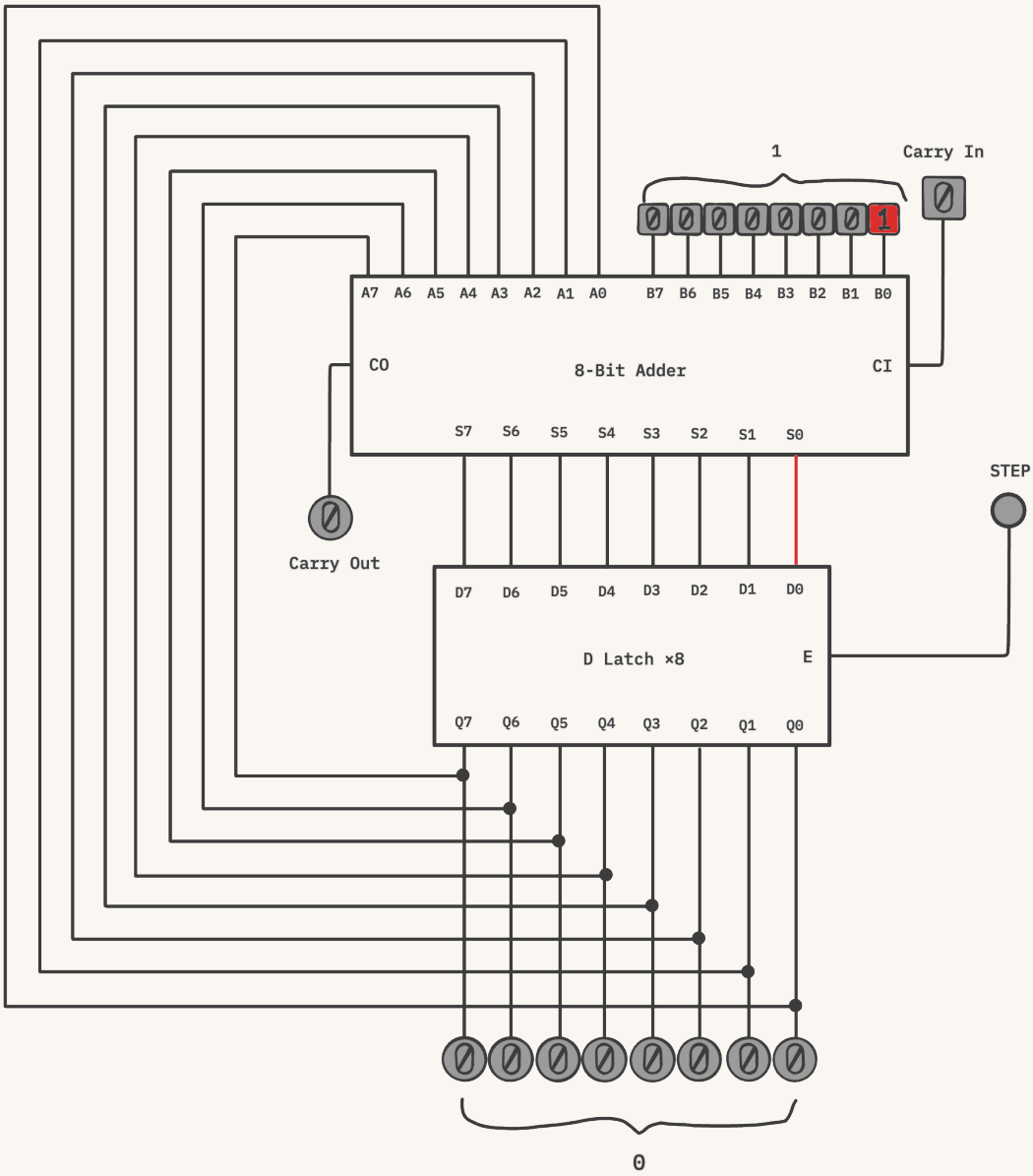


Diagram 7.2. D latch.

But we have a problem. Let's say we now try to use 8 of these D latches to hold the result from our adder, which would then feed back into the input for our accumulator. It still wouldn't work.

Here is the problem: say we have the enable wire hooked up to a button. When that button is pressed down, the enable wire is on, so `Q=D` for that time. But if `Q` feeds back into the adder, and the result of the adder `D` changes quickly enough, `Q` can change again, jumping unpredictably based on how long we hold that button for.

If we want the accumulator to work correctly, we need the enable wire to turn on for an instant and then turn back off. That is just hard to do.



Full 7-frame animation at bytecolony.computer/writing/the-cpu/#f=d-latch-accumulator:1.

Diagram 7.3. D latch accumulator.

As you can see in this diagram, even pressing the button quickly jumps the result up by 5. With real transistors, even if you try to physically tap the button, it could count up by millions, overflowing these 8 bits thousands of times.

How long you hold the button decides the answer. It doesn't count in ones.

But what if we had a storage circuit that only copied `D` into `Q` at the exact instant `E` turns on?

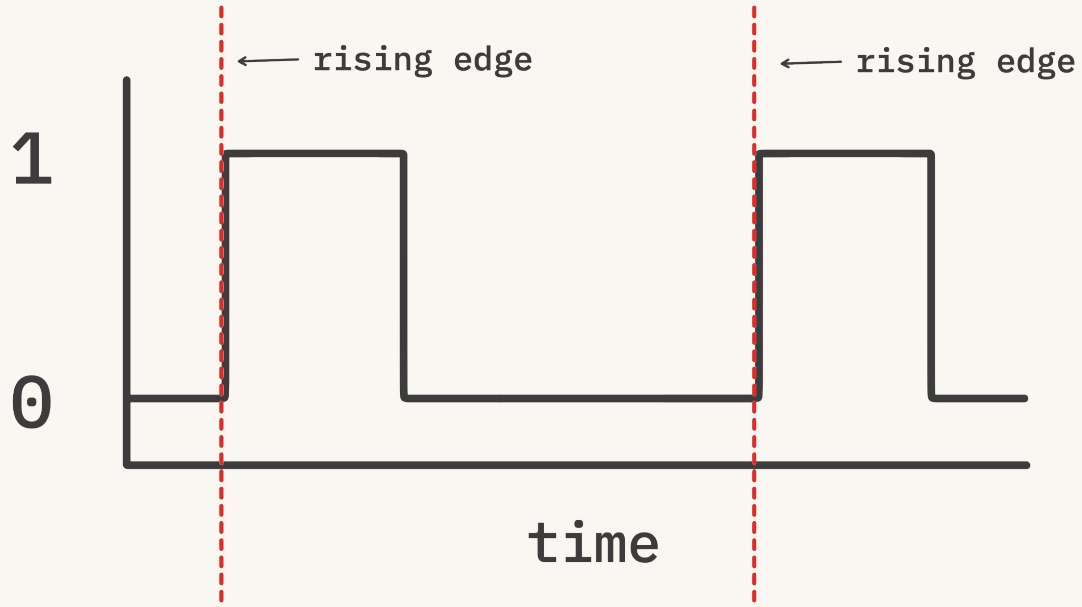


Diagram 7.4. The rising edge of a signal.

This graph shows the state of a wire. When the line is at the top, it is on. When it is at the bottom, it is off.

When a switch is flicked or a button is pressed, a transition happens. That transition is called an "edge", and when the wire turns from off to on, it is a rising edge.

Now what if we only set `Q` to `D` on that transition, at the rising edge? The edge is an instant of time, not a duration.

The circuit that does this is called a D-type edge-triggered flip-flop. This might sound like a mouthful, but D-type just means it takes in a data input, edge-triggered means it triggers on the edge of a signal, and flip-flop means it is a storage circuit similar to a latch, but usually edge-triggered.

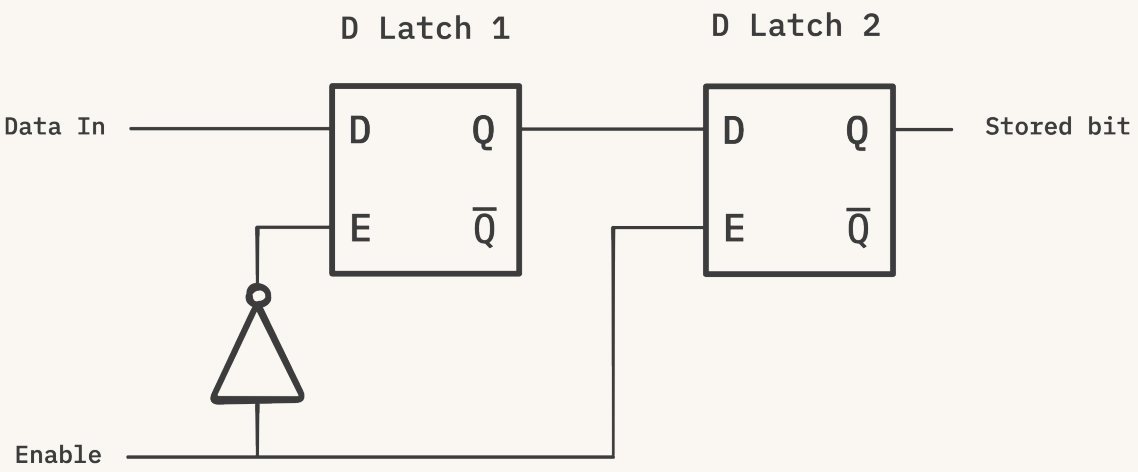


Diagram 7.5. A flip-flop.

How it works is, when the enable wire is off, the first latch mirrors `D`. That is because the NOT gate flips the enable signal, so the first latch sees it as on.

Then when enable turns on, the second latch stores the output of the first one. And because enable is now on, the first latch is locked, so it can't change!

So if `D` changes while enable is off, we are all good because the second latch is locked. But if `D` changes while enable is on, we are fine because the first latch is locked.

Here is one storage cell, which is just the flip-flop we showed above:

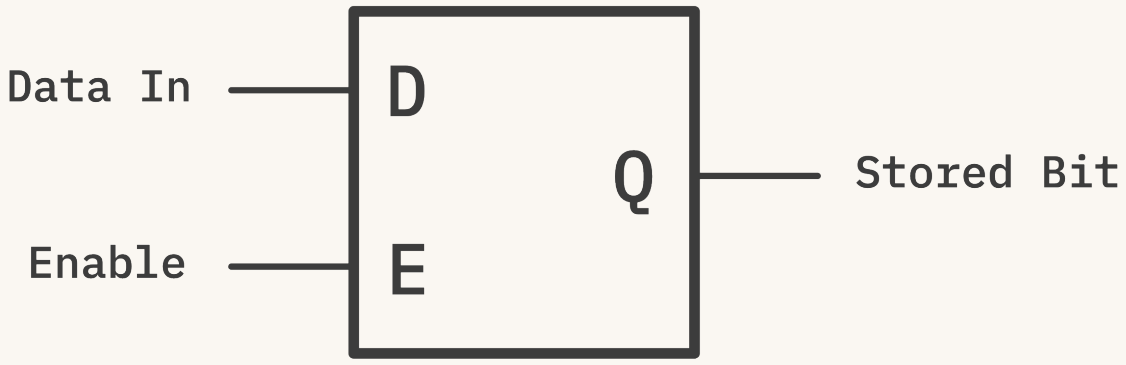


Diagram 7.6. A one-bit storage cell.

If we connect 8 of them side by side, we get one byte of storage:

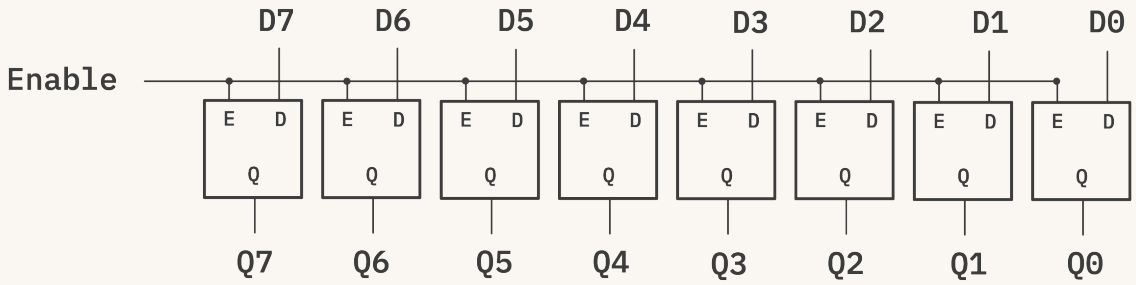


Diagram 7.7. Eight storage cells.

And we can put all that into a chip called an 8-bit register:

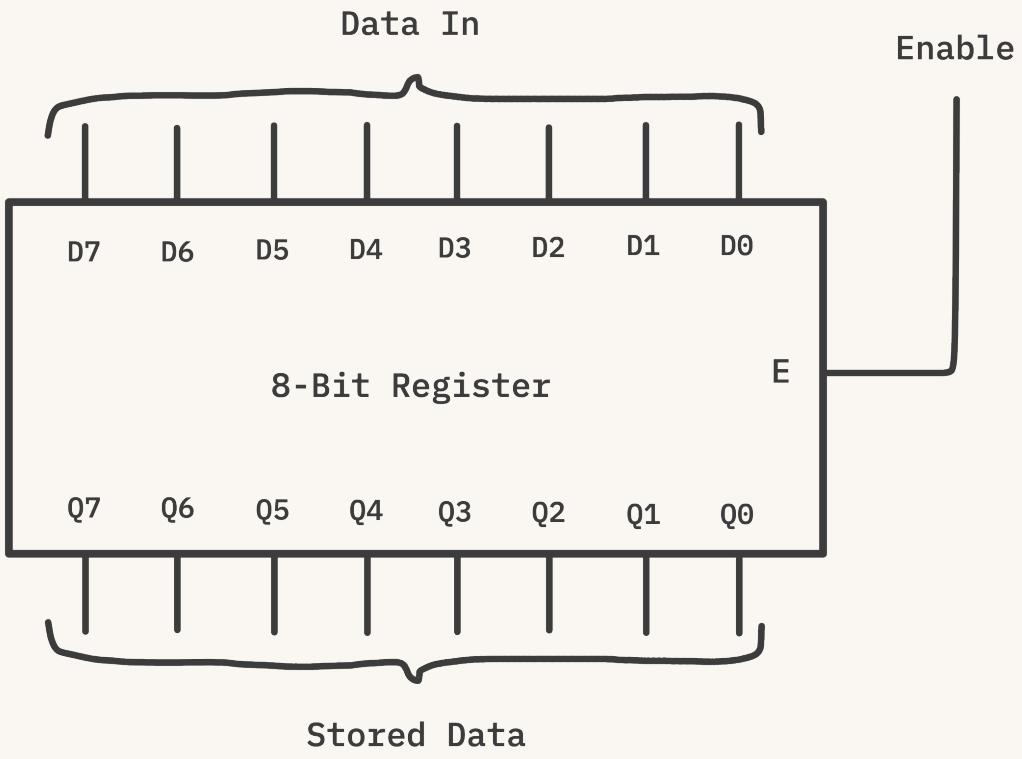


Diagram 7.8. An 8-bit register.

Now with this register, let's build a basic accumulator/adder circuit.

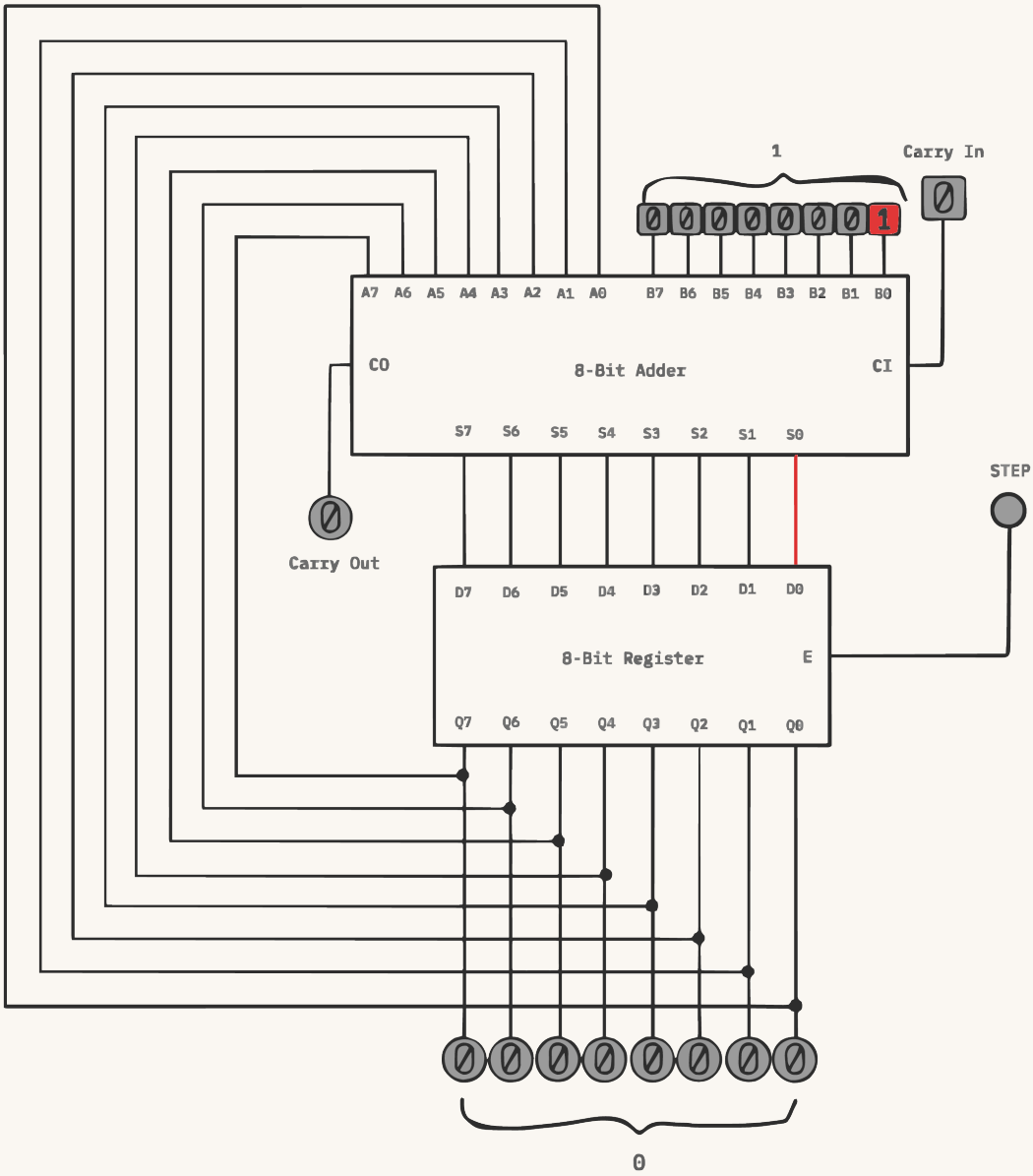


Diagram 7.9. Our full accumulator.

As you can see, the circuit kindly waits for us, and is incrementing by ones!

How this works is, when the `STEP` button is pressed, the output from the adder gets saved into the register on the rising edge of that press. This then changes the input to the adder, which changes its output, but the register holds its value because it only captures on the edge of the press. Holding `STEP` down does nothing special. So each press increments the register's value by, in this case, 1.

Now, a real computer would need to do these kinds of things millions and billions of times per second, and we don't have some human clicking a step button. What we have is a circuit that automatically goes on, off, on, off billions of times per second. This is called a clock. Each rising edge of the clock acts like one press of `STEP`.

Here is the basic concept of a clock:

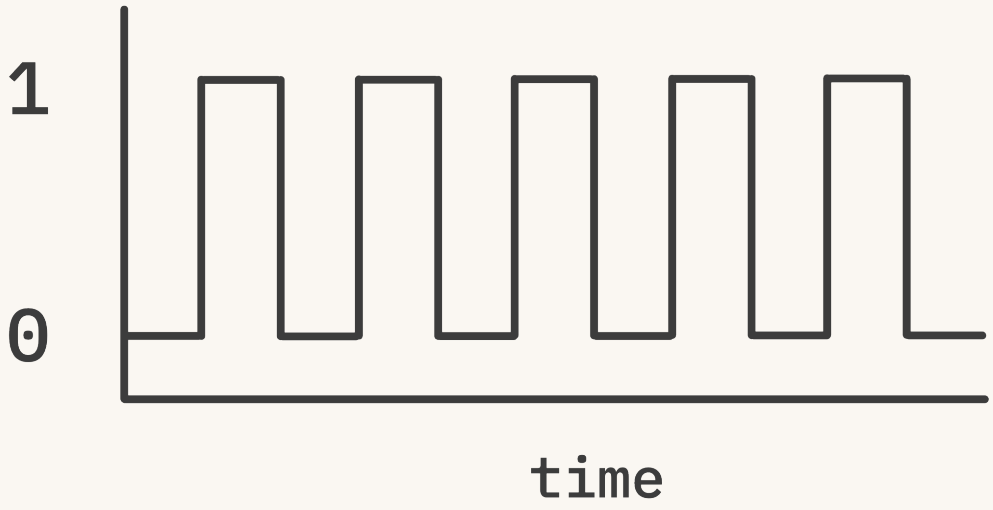


Diagram 7.10. A clock signal.

This repeating on-off behavior can be achieved in different ways. A rough toy example is feeding the output of a NOT gate back into its input, so the signal keeps trying to flip back and forth between on and off.

Real clocks are built in more sophisticated and reliable ways, often using crystals or other oscillator circuits. But we do not need to build the clock itself here. For now, we can treat it as a little chip that repeatedly produces the same on-off signal.

Just imagine the new accumulator with a clock signal instead of a `STEP` button. I am too lazy to draw it for you.

We now have some storage. A register that can hold a byte, and update exactly when we want.

But registers on their own are not enough. We need to be able to move numbers between registers, the adders, and the main memory, which we will build later.

2.2. Buses

One simple solution to move bytes around would be to give every component its own bundle of 8 wires to every other component, but that would become a mess very quickly.

A simpler solution is to have one single 8-bit data highway, where components can put data on and take data off. This collection of 8 wires is called a bus.

One more thing, moving forward when I want to draw a collection of 8 wires, instead of drawing each wire, I will just draw a thick arrow that represents 8 wires.

To show the state of the wires, I can write a number in the arrow; the number 0 for example means the wires are all off, and the number 2 would mean the wires are `00000010` which is 2 in binary.

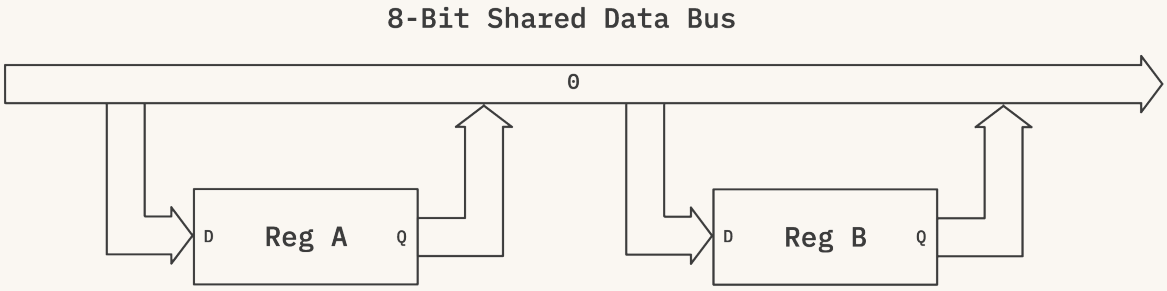


Diagram 8.1. Two registers sharing a bus.

But we have an issue: this diagram is technically not possible yet. Say register `A` is outputting `00000000` and register `B` is outputting `00000001`, both onto the same 8 wires. Look at the last wire. `A` is driving it low, so that wire is connected to `-`. `B` is driving the same wire high, so it is also connected to `+`. What happens?

Yep. A short-circuit.

We need a way to connect these registers to the bus, but also let them get out of the way when they are not supposed to actively drive a wire to `-` or `+`, like I discussed previously.

Just setting the output wires to `00000000` is not enough. On a shared bus `00000000` is not nothing. It is actively driving the bus to `-`.

One clean way to solve this problem is by using something called a tri-state buffer. It has two inputs, `E` and `D`, which stand for enable and data.

We have seen `E` before, in the context of "enable writing" but now we are using `E` in the context of "enable outputting".

If `E` is on, the output will just be whatever `D` is, so either `0` or `1`. If `E` is off, no matter the value of `D`, the output will be `Z`.

`Z` means the buffer's output is disconnected from the bus. It is not driving the bus to `+` or `-`, so another component can safely drive the bus without a clash or short-circuit.

Or in other words, this buffer is not touching the wire. `0` is very different: the component is actively pulling the wire down. So we have three states:

```
1: driven high
0: driven low
Z: disconnected
```

This relay diagram of how a tri-state buffer works should make this concept crystal clear.

Also I have drawn everything the output wire is currently touching in yellow. Yellow is just there so you can follow the path with your eyes; it doesn't mean anything.

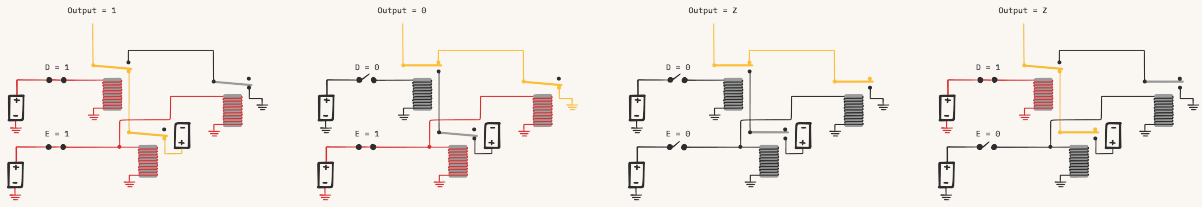


Diagram 8.2. A tri-state buffer built with relays.

This looks complicated, so let me break it down.

First, ignore the two relays on the right and look only at the `D` relay at the top. Its arm is attached to the output wire, and it works just like the output driver from before. When `D` is `1`, the arm is pulled down onto the wire that leads toward the battery, `+`. When `D` is `0`, the arm goes up onto the wire that leads toward ground. Remember, ground is just the `-` side.

The important idea is that neither of those wires is directly connected to `+` or `-`. Each one has a relay between it. Both of those relays are controlled by `E`.

When `E` is `1`, both gaps close. The output is now connected to whichever side `D` picked, so it is driven to `1` or `0`.

When `E` is `0`, both relays touch a point connected to nothing. Both wires lead to a dead end. The output wire is touching nothing. That is `Z`.

So we have three states:

E	D	Output
1	1	1
1	0	0
0	0	Z
0	1	Z

This is the logic gate diagram for a tri-state buffer:

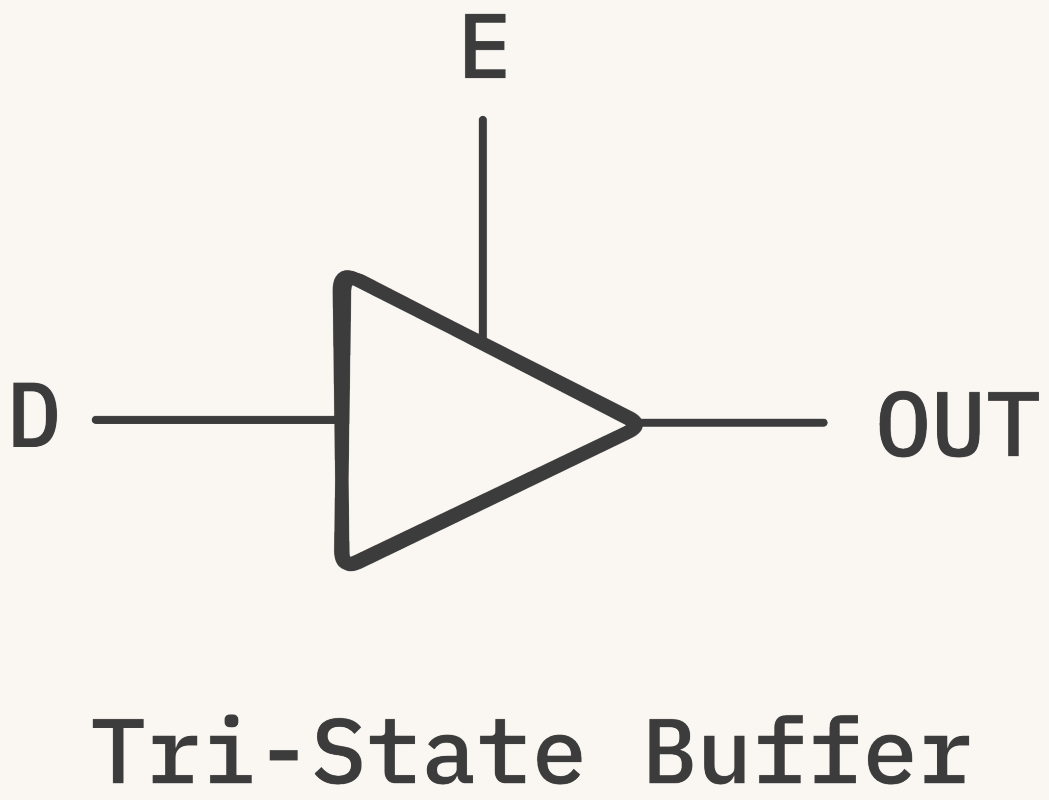


Diagram 8.3. A tri-state buffer logic gate.

Now let's address this enable conundrum. We now have two uses for the word enable, with completely different meanings and contexts. One means enabling writing, and the other means enabling output. From now on, we will use two separate terms to avoid confusion: `WRITE` and `OUT`.

So we can make a new type of register, one with `WRITE`, `OUT`, `data`, and a `Q` output that shows the stored bits 0-7. By `data`, I simply mean the input data that we can store when `WRITE` is enabled.

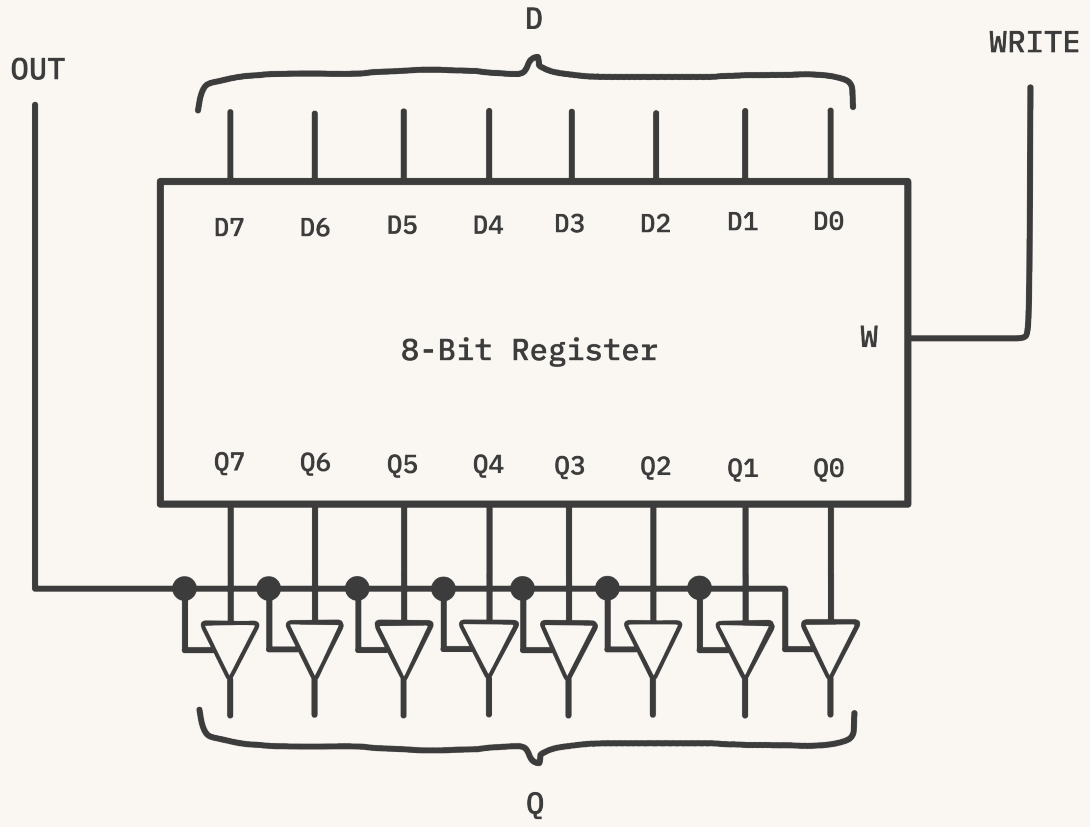


Diagram 8.4. Our register with an OUT input.

We are just connecting `OUT` to all of the enables in the tri-state buffers. So if `OUT` is `0`, `Q` will be all `Z`, and if `OUT` is `1`, `Q` will be whatever is stored in the register.

With that, we can use these new registers with a common bus to move data.

Here is an example where the content of register `A` gets copied into register `B`.

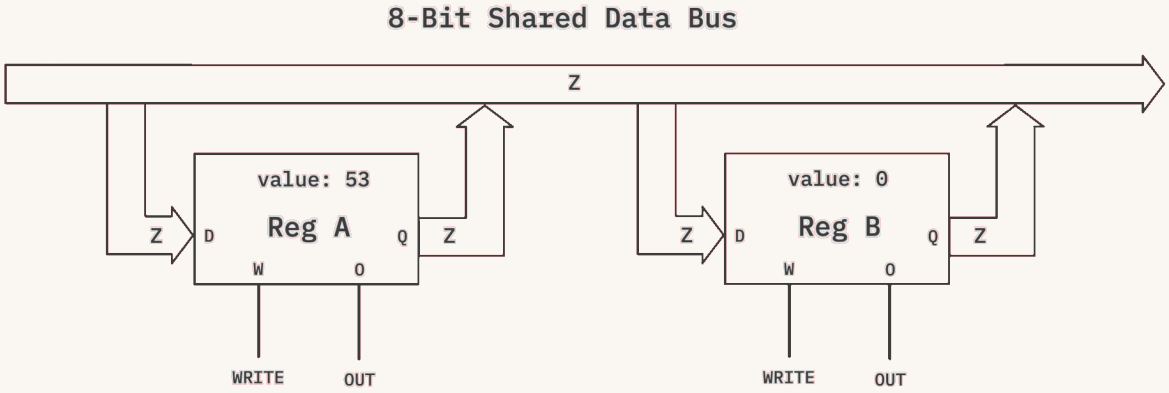


Diagram 8.5. Copying register A into register B through the shared bus.

I have some text inside the register that shows what it is storing. We of course have `W` and `O` which are `WRITE` and `OUT` as well as `D` and `Q` which are the inputs and outputs.

Of course, on the second frame, when `OUT` of register `A` is enabled, the `D` wires of both registers are also going to be 53 because they are directly connected to the bus.

Also generally in this diagram, register `B`'s output is sometimes shown as `Z` even when the bus is 53. That is because register `B`'s `OUT` is off, so register `B` is not driving the bus. It may be connected to a bus currently at 53, but the 53 is coming from register `A`. So technically, those wires are at 53 but… it just looks better to keep them at `Z`.

By the end of this sequence, we have copied the value 53 to register `B`! We can have many more registers sharing a common bus, as long as only one is driving the bus at a time.

Now we can store a byte, compute a sum, and move bytes around!

The next problem is organization and scale. How do we organize many stored bytes so the machine can choose one slot, read it, and write back to it? A handful of registers aren't enough.

2.3. Organizing Data

We want to build a system that organizes data into a simple structure.

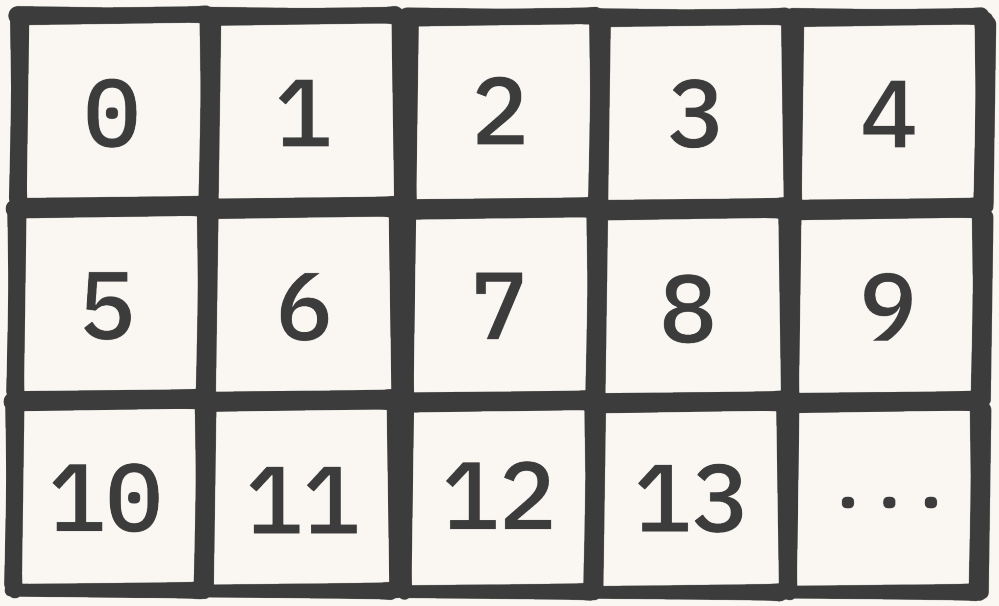


Diagram 9.1. Our data structure.

Many slots, each with its own address.

This system is known technically as RAM: Random Access Memory. It is called RAM because when the CPU wants to access a slot, it just knows the number and can access any slot at

will. It is not like flipping through a book looking for the right page. It is more like grabbing a book from a bookshelf, where you already know exactly where the book sits.

Now let's think about exactly what we would want this `RAM` chip to do.

- `address`: the slot we wish to access
- `WRITE`: whether we want to write a value to this address
- `OUT`: whether we want to output the value onto the bus
- `data in`: the value we would like to write
- `data out`: the data output line

To be clear, `WRITE` and `OUT` are control signals, so just 1 input wire each.

For this demo `RAM`, `address` is only 4 input wires. `data in` and `data out` carry bytes and are both connected directly to the common bus.

This only works if no other part is driving the bus when `OUT` is enabled.

So we are going to build a minuscule 16-byte `RAM`: 16 addresses, with each address storing one byte. This design can be scaled up easily.

Our address will be 4 bits long, because `2^4` is 16.

We could do this as a tall stack of 16 registers, but a grid is nicer.

So we will split the 4-bit address in half. The bottom two bits pick the row, and the top two bits pick the column:

```
top 2 bits    = column
bottom 2 bits = row
```

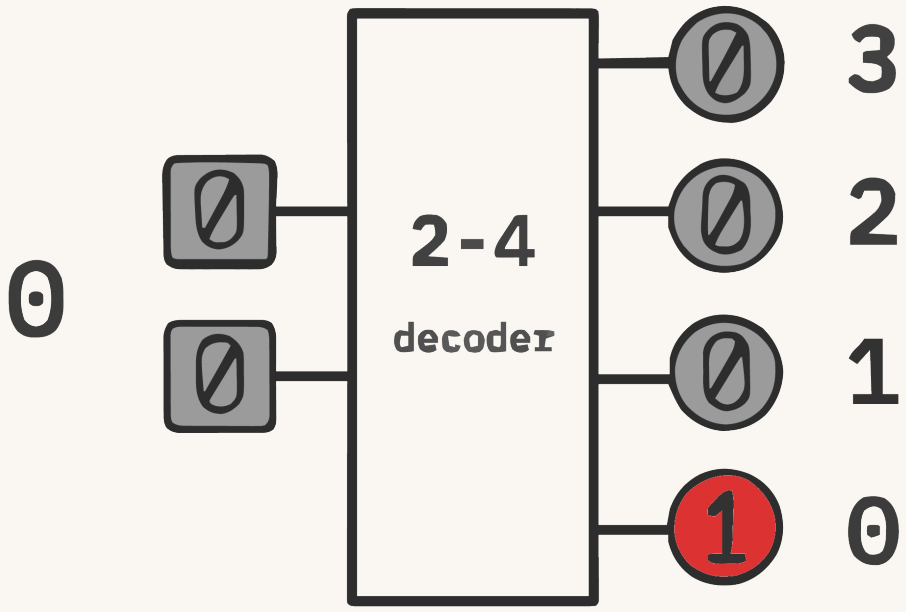
Two bits can choose 4 values, so this gives us a 4×4 grid of memory slots. That is 16 total bytes!

Once the address selects a slot, two things can happen:

- If `WRITE` turns on, the selected slot stores `data in`.
- If `OUT` is on, the selected slot drives its stored byte onto `data out`.

Let's start with building a simple decoder. This decoder will take 2 bits of our address and, based on that number, turn on exactly one out of 4 wires.

In the diagram the top bit is the bigger bit, the 2's place, the bottom is the smaller bit, the 1's place.



Full 8-frame animation at bytecolony.computer/writing/the-cpu/#f=2-4-decoder:1.

Diagram 9.2. How a decoder works.

As you can tell, no matter the inputs, exactly one output wire is on at a time.

We use one `2-4` decoder for the rows and another `2-4` decoder for the columns. Where the selected row and selected column cross, that is the byte we want to target.

This diagram shows a few addresses as examples. Each address gets its own little intersection.

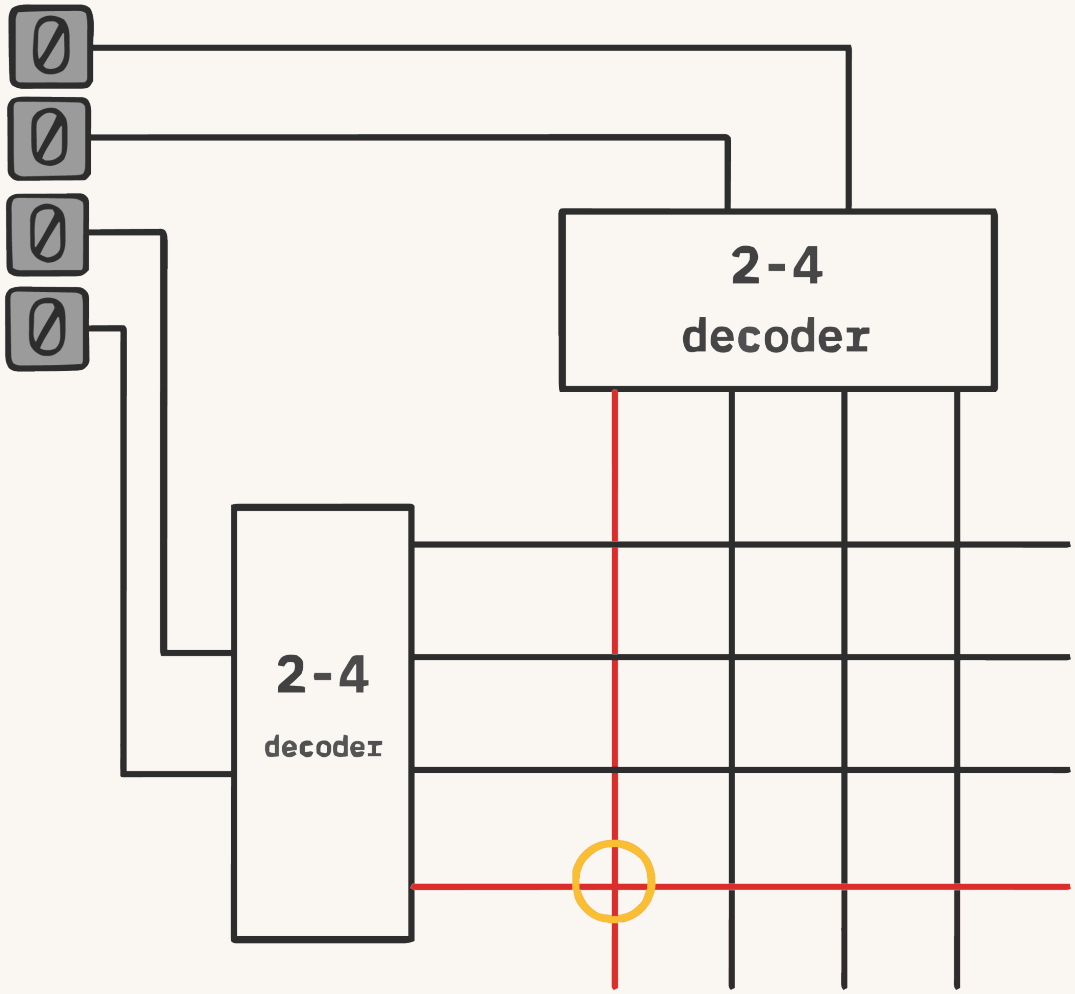


Diagram 9.3. Where the row and column meet.

How a decoder works is extremely simple. It just uses a bunch of logic gates to ask these simple questions.

- If `00` -> turn on wire 1
- If `01` -> turn on wire 2
- If `10` -> turn on wire 3
- If `11` -> turn on wire 4

Here is how it works if you care:

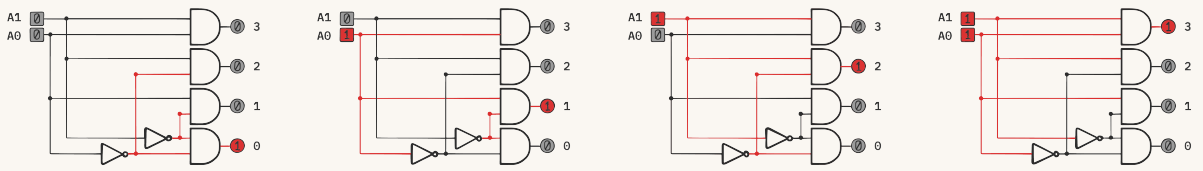


Diagram 9.4. 2-4 decoder internals.

Honestly? That's it. We can use two decoders, sixteen registers, some wires and buses all mashed together with some extra logic gates and BOOM! We have some `RAM`.

In this diagram, blue lines are 8-bit data buses. `OUT` is yellow, and `WRITE` is orange. They are still ordinary wires; the colors are only there to make the diagram easier to follow.

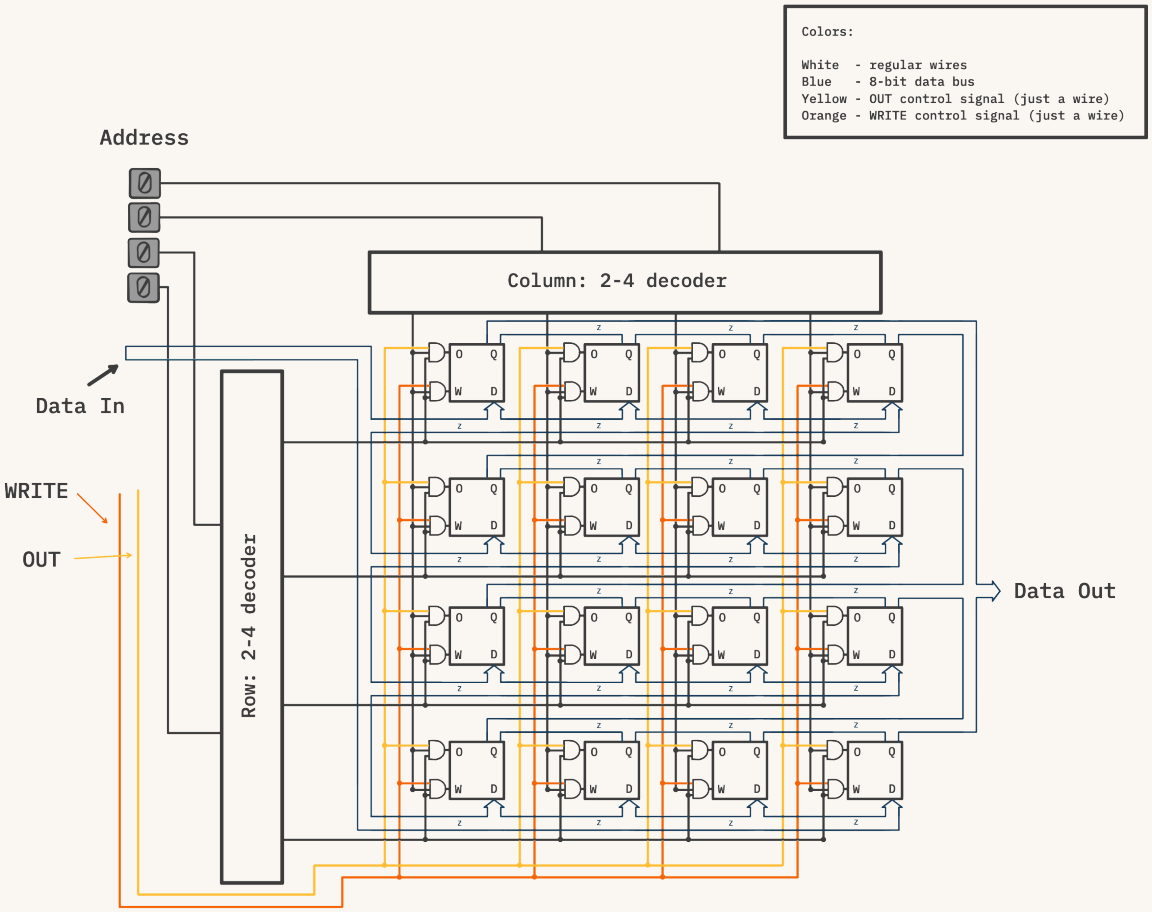


Diagram 9.5. A zoomed out RAM diagram.

This is kind of a lot to unpack, so let me explain the high level parts before we zoom in and take a closer look.

We have a blue data in bus that is fed into the bottom of all of the registers. We also have another blue data out bus that comes out of the top of all the registers and combines into one output. Every slot is connected to both buses, but only the selected slot is allowed to use them. If the slot's row and column are selected, it can either read from `data in` when `WRITE` is on, or drive `data out` when `OUT` is on.

Let's look at one cell more closely:

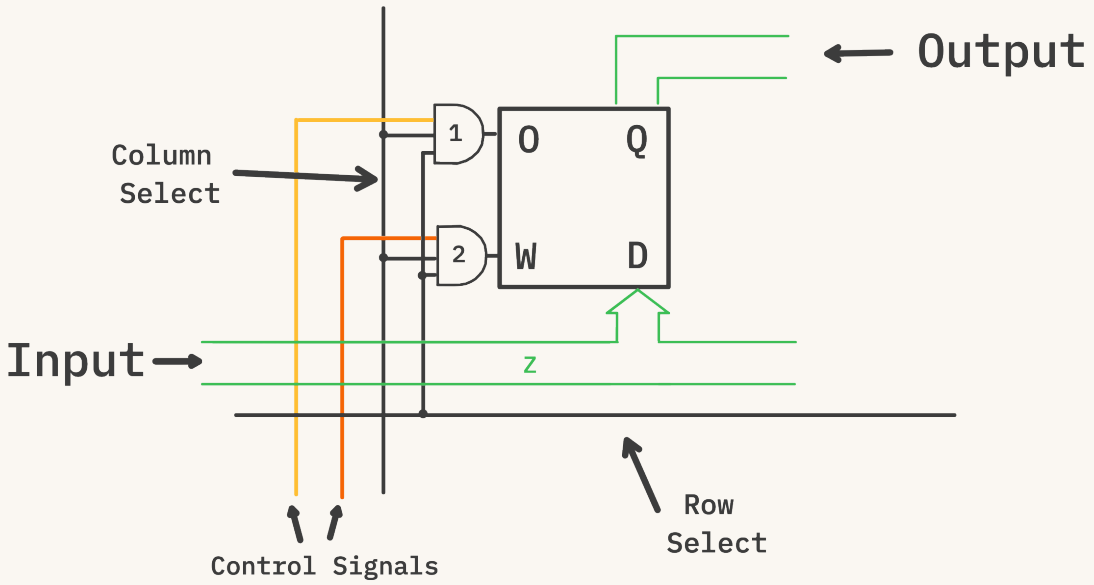


Diagram 9.6. A zoomed in RAM cell diagram.

What AND gate 1 checks is, if `Row Select` and `Column Select`, and `OUT` is on, then that means we have selected that register to output its value, thus we turn on `OUT` and the register will output something on the `Output` bus.

AND gate 2 checks, if `Row Select` and `Column Select`, and `WRITE` is on, then that means we have selected that register to write to, thus we turn on `WRITE` for that register, and it will write the data on the `Input` bus.

So yea, both AND gates take in 3 inputs, if you are wondering how that works, just think of two AND gates chained together.

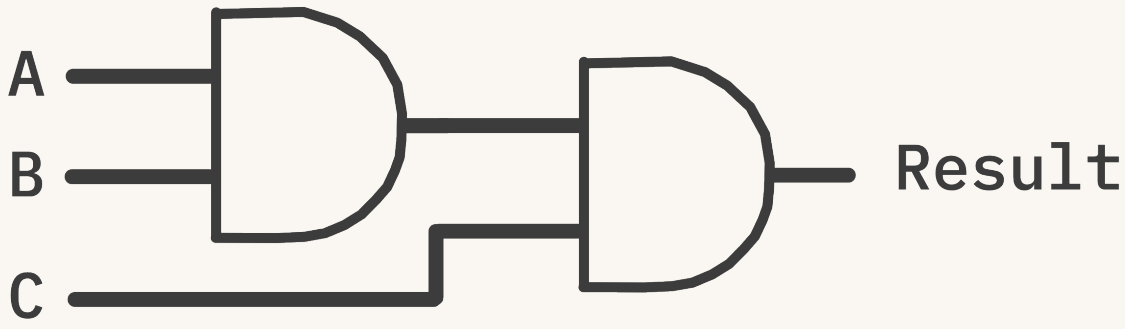


Diagram 9.7. A three input AND gate.

So now that we have built `RAM`, let's pretend that instead of 16 registers, we have a `RAM` array with 256 registers. The same logic can be copied, just with two `4-16` decoders instead of two `2-4` decoders and 8 address inputs rather than 4.

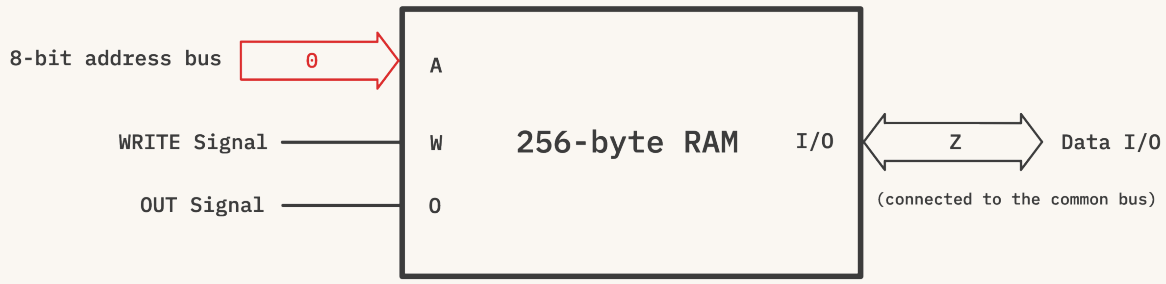


Diagram 9.8. Our RAM chip.

Technically, there are still two buses inside the `RAM`: a data-in path and a data-out path. That is basically what we saw with the register in the bus section.

But drawing two separate data buses every time is cumbersome. From the outside, we can abstract this as one shared data bus with a double-headed arrow called `I/O`, which stands for input/output.

It is practically just like having two buses, one for input, one for output.

If `W` is on, `RAM` copies the value from the data bus into the selected address.

If `O` is on, `RAM` drives the selected address's value onto the data bus.

So from now on, instead of drawing registers connected to a common bus like this, where we have a separate `D` and `Q`, we can just draw them like this:

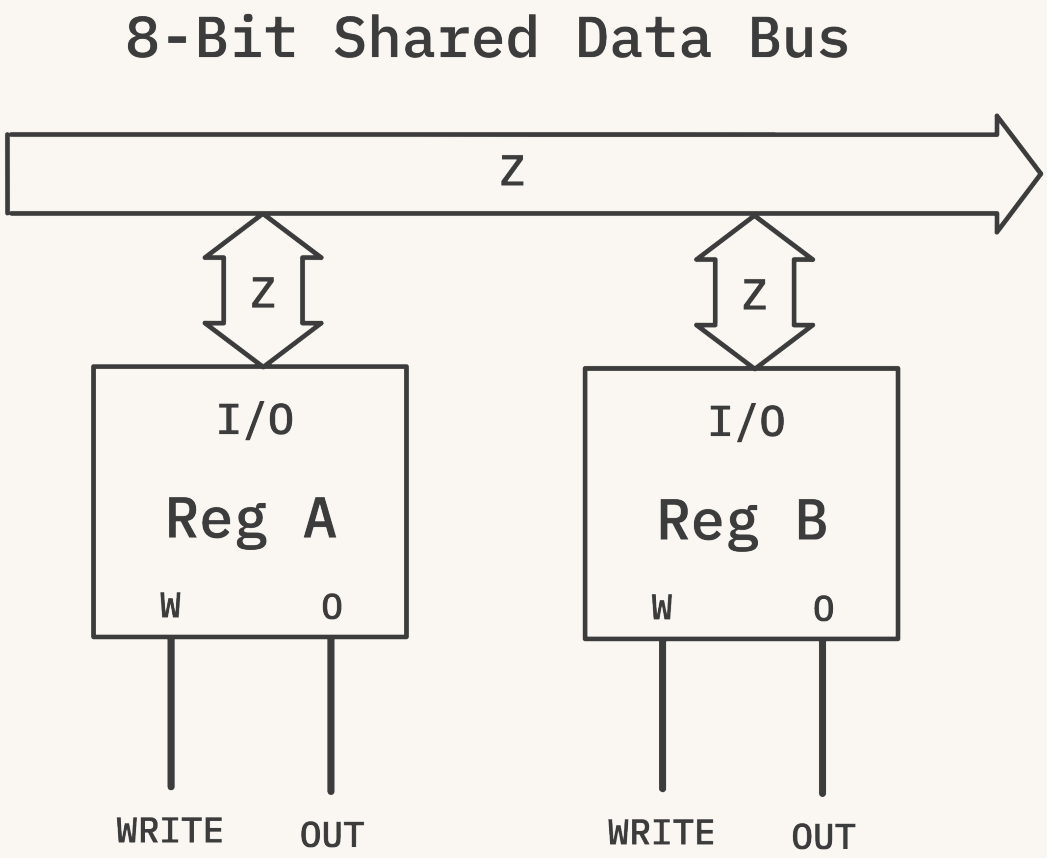


Diagram 9.9. I/O Registers.

They both are the same technically, just this is easier to draw, so moving forward, instead of drawing two buses for `D` and `Q` I'll just draw one double-headed `I/O` bus.

Now back to the `RAM` chip.

If you pay close attention to the diagram, you will notice that the address input is not directly connected to the common data bus.

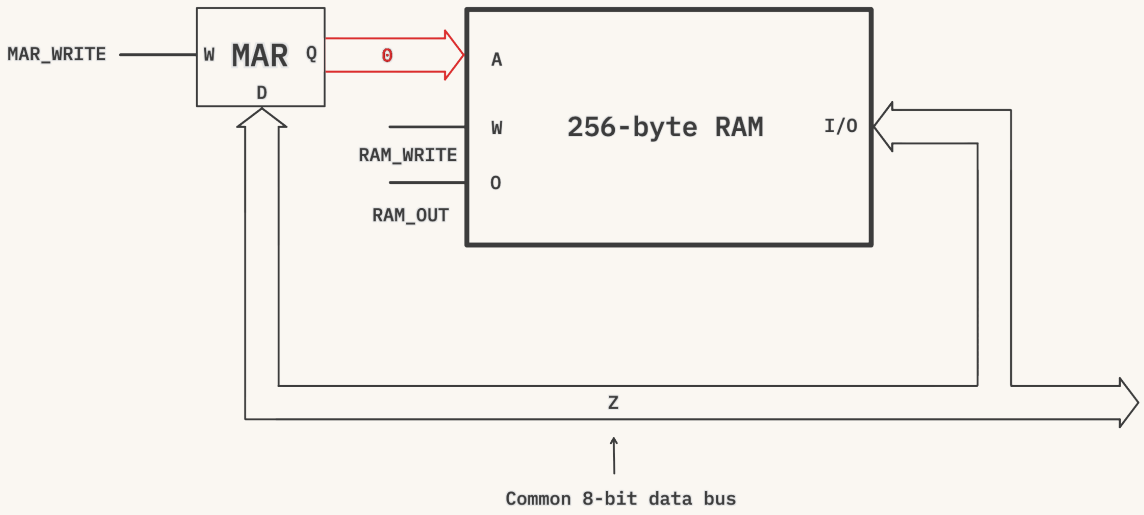
That is intentional. The data bus is for moving values around the CPU. During a `RAM` operation, it needs to carry the value being written to `RAM` or the value being read from `RAM`. So it cannot also keep holding the address at the same time.

We need something to hold the address while the data bus is being used for the actual I/O.

This is called the Memory Address Register, or `MAR`.

The `MAR` is just a regular 8-bit register with no `OUT` control signal as it is always outputting directly into `RAM`.

We first put an address on the data bus and turn on `MAR_WRITE`. The `MAR` stores that address. Then the `MAR` keeps sending that address to `RAM`, leaving the data bus free to carry the value being read or written.



Full 7-frame animation at bytecolony.computer/writing/the-cpu/#f=mar-ram-demo:1.

Diagram 9.10. How the MAR works.

So first, we put 28 onto the common bus. Enable `MAR_WRITE` and store that into the `MAR`. We then remove 28 from the common bus and enable `RAM_OUT`, and we get 6 as the value stored in slot 28. Cool.

2.4. The ALU

If we have a handful of registers and `RAM`, we can now move bytes around using this common data bus. But what we really need is a component that can "process" numbers, a component that can do arithmetic and logic. Thus we have "The Arithmetic and Logic Unit," or `ALU` for short.

Imagine a chip where we could input two numbers, an operation, and output a result, along with some other information.

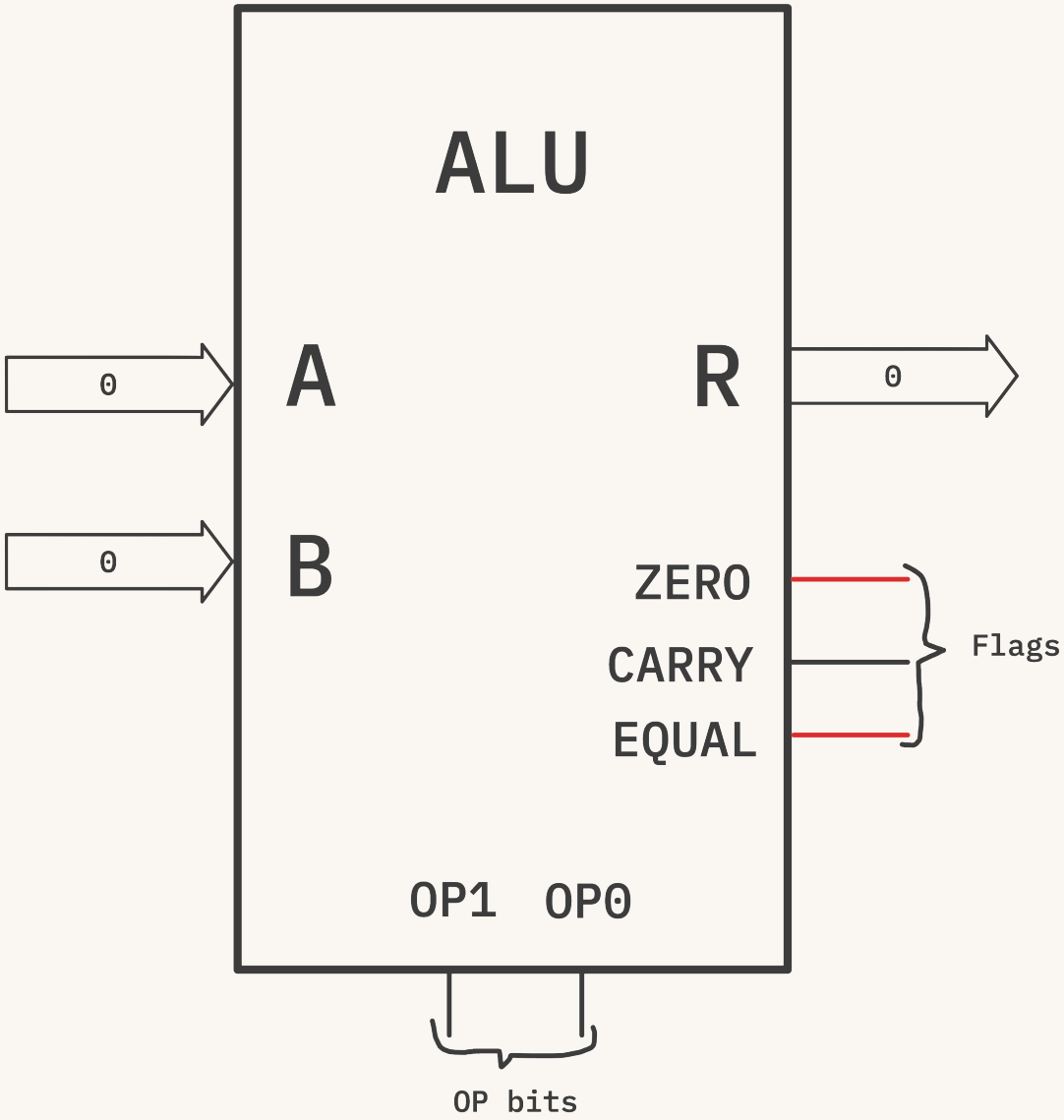


Diagram 10.1. The `ALU` chip.

For this CPU, I am keeping the `ALU` simple. It will have two operation-select bits, which gives us four possible operations:

`OP`	Operation	Meaning
`00`	ADD	output `A + B`
`01`	AND	output `A AND B`
`10`	OR	output `A OR B`
`11`	XOR	output `A XOR B`

The `ALU` will also output a few flags, which are just extra yes/no facts about the result or the inputs:

Flag	Turns on when
`ZERO`	the result is `00000000`
`CARRY`	addition spills past 8 bits
`EQUAL`	`A` and `B` are the same

So, in the previous diagram, we did 0+0 which is 0, so the `ZERO` flag is on, and the `EQUAL` flag too because both inputs are equal.

This next diagram uses a new component. It is a mix of two registers we have already seen.

Remember the D latch, the first storage circuit we built? While its enable was on, `Q` simply equaled `D`. No edge-triggering or anything like that. This new register is a D latch with the enable permanently on: it is always storing whatever value is on its input.

But like our newer registers, its output goes through tri-state buffers with an `OUT` control signal, so we still decide when it outputs.

So in total: a data input that is always being stored, an `OUT` control signal, and an output.

Also, you'll see me feeding two buses into a single logic gate. "How does that work?", you might think. Well, there are really just eight gates, one per bit. One gate takes `A0` and `B0`, the next takes `A1` and `B1`, and so on. Eight output wires, which is just another bus.

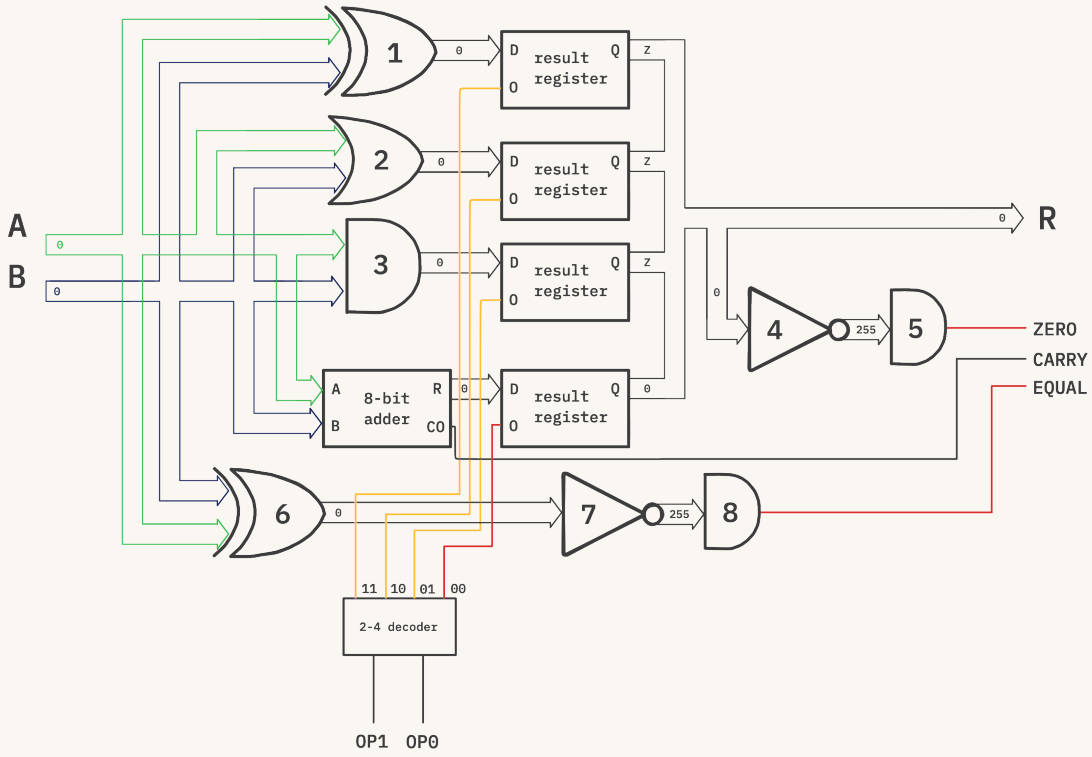


Diagram 10.2. `ALU` internals.

At its crux, the `ALU` works by routing `A` and `B` into all 4 operations at once, in this case XOR, OR, AND, and ADD. We store each of the results in a corresponding result register.

To decide which one to output, the `OP` bits go into a `2-4` decoder that enables exactly 1 of the result registers onto the `R` bus.

Now for the flags.

First, the `ZERO` circuitry. It consists of:

`NOT -> AND`

The NOT gate on the right, labeled 4, is just like before: there are actually eight NOT gates, each flipping one wire of the bus. So it takes in a bus, and outputs a bus.

But the AND gate next to it, labeled 5, takes in a bus and outputs just one wire. Like the three-input AND from the `RAM` section, this AND gate takes in 8 inputs and produces 1 output. It just checks if all of its inputs are on.

So if we flip each bit, and then check if all of them are on, we get the `ZERO` flag. This makes sense because all the bits going into the AND gate can only be on if they were originally `00000000`, a.k.a. zero!

Next, let's cover the `EQUAL` circuitry. It consists of:

`XOR -> NOT -> AND`

The XOR gate, labeled 6, is basically like last time: we XOR each pair of bits from `A` and `B`, and create an eight-bit bus. Remember, XOR outputs `0` when its two inputs are the same. So if `A` and `B` are equal, we get `00000000` as the output.

Then we flip the bits with gate 7, getting `11111111`, and if we AND them all together with gate 8, we can check if they are all true. If even one pair of bits differs, that wire ends up `0` after the flip, and the AND outputs `0`. Simple.

The `CARRY` flag is simple: we just connect the adder's carry-out, `CO`, straight out. Of course, it only means anything when we are actually adding.

2.5. The Big Picture

Here is a big picture diagram of the whole CPU.

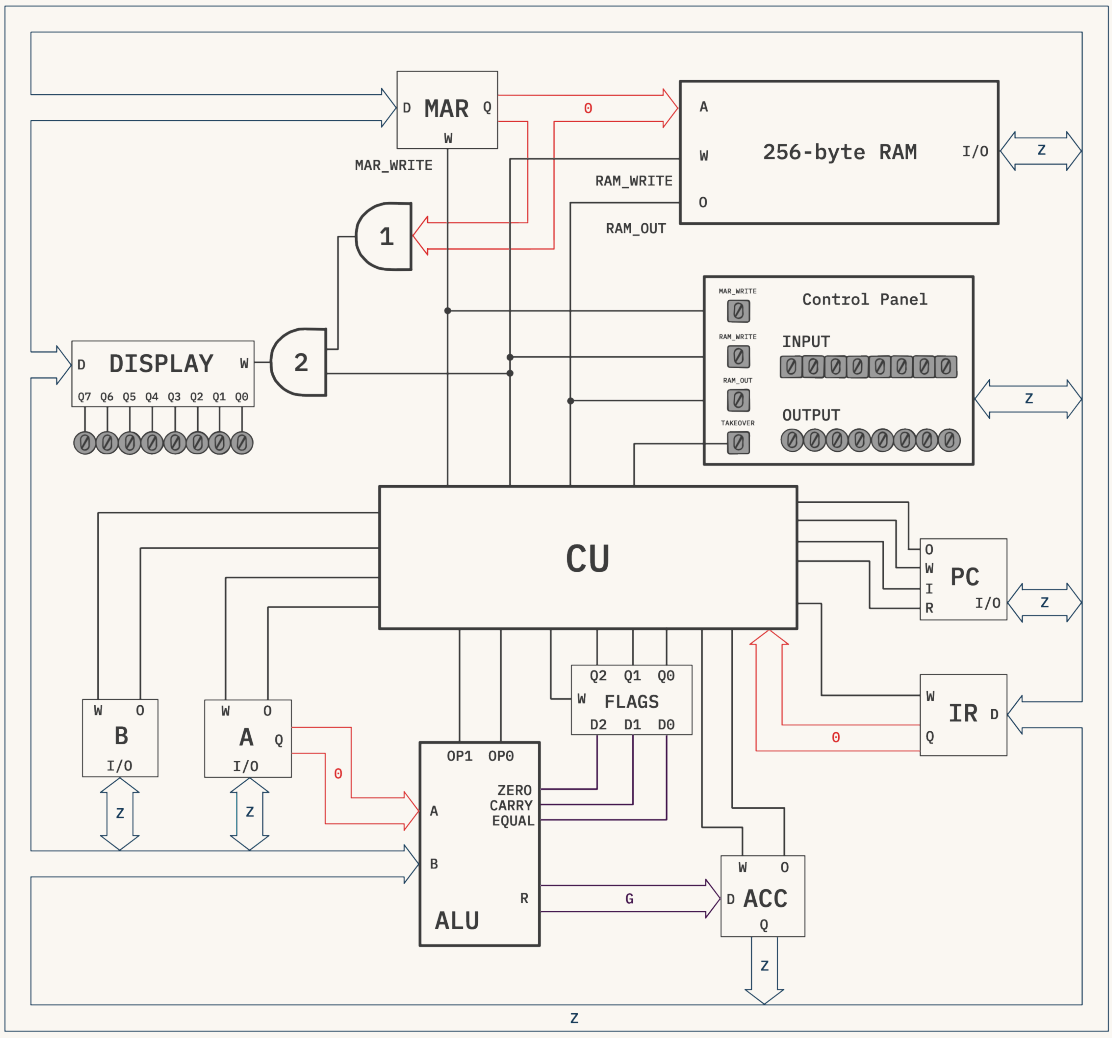


Diagram 11.1. The whole CPU.

Hey? Doesn't this diagram look familiar? Well, yes, we have come full circle from the intro, except this time the CPU isn't something completely foreign. Of course we still have a lot to learn and build, but wow. Just wow.

Some quick notes on the diagram before we dig in.

The 8-bit data buses are blue when not being driven, just like before. If a control wire is red, it means it's on. If a bus is red, it means it's being driven like normal. But we have something new here that we haven't seen before.

Purple wires and buses mean garbage. This is not the same as `Z`. A `Z` wire is one that nothing is driving. A `G` wire is being driven, with an actual value on it. That value is just nonsense, hence the name garbage. So where does that garbage value come from in this diagram?

Remember, the `ALU` never stops computing. It always takes its inputs and has a result instantly. One input comes from register `A` (we will talk more about how register `A` works later), but the other input comes from the bus, and when nothing is driving the bus, the input is sitting at `Z`. The thing is, in this CPU an input at `Z` would just behave like 0. Our

gates are built from relays, and a relay coil with nothing driving it is simply off, exactly as if you fed it 0. So really, the `ALU` would be computing `A + 0` if the second input was `Z`.

But we mark it as garbage anyway, because in a real CPU made from transistors instead of relays, a `Z` value does not settle to a clean 0; it drifts and can get really funky. That is why we treat those values as garbage: we just ignore them. If the bus is being driven then we can of course use those values.

Now back to the diagram.

Almost every single chip here, we have already built. The `RAM` and `MAR` combination, we have seen how that works previously. `A`, `B`, `ACC`, `IR`, `FLAGS`, and `DISPLAY` are all just slight variations of registers. `PC` is an accumulator style circuit, and we know how `ALU` in the bottom left works.

Let's examine each element of the CPU closely and see what details changed and why.

First, register `B`. Register `B` is a normal tri-state 8-bit register with `W` and `O` control wires. We will talk about its use in more detail later.

We then have register `A`, which is just like register `B`, but it also has a `Q` output in addition to `I/O`. This `Q` output is simply the value stored inside the `A` register, bypassing the tri-state buffers and feeding directly into the first input of the `ALU`. So even though the output (`O`) control signal is off, that `Q` is still driving the `ALU`'s first input, in this case to 0.

Next we have the `ALU`. Its second input comes from the common bus, so the other number it works with is just whatever is on the bus at the time. Other than that it is as normal, but the flags are all going into a register called `FLAGS`. This register is just a 3-bit register: it only stores 3 bits. These three bits are just the values for each flag. This 3-bit register has no `O` control signal, thus no tri-state buffers, so it is always outputting.

Then we can see the result of the `ALU` operation is being fed into another regular 8-bit register called `ACC`. It is slightly different to register `B` because its input doesn't come from the common bus, but from the `ALU`, so it has two separate `Q` and `D` instead of just one `I/O`. It still has the regular `W` and `O` control signals though.

Register `IR` is a simple register without an `O` control signal. It just takes input from the bus, and outputs it into the `CU`. We will talk about what exactly the `CU` is and the jobs of these different parts a little later on.

Next we have `PC`, which is a combination of an adder and a register, something like what we have seen previously. The register part of `PC` has its regular control signals `W` and `O`, and the `STEP` button from before has been replaced with an `I` control signal, which stands for increment. We also have the `R` signal which just resets the register back to 0 with some more logic gates and wires. Nothing too fancy.

Our `MAR` and `RAM` are the same as before, but if all of the bits stored in `MAR` are 1, meaning the address is 255, then that first AND gate in the diagram, labeled 1, turns on. We then AND that with `RAM_WRITE`: if we are writing to address 255, we also write into the `DISPLAY` register.

What ends up happening is that `DISPLAY` stores whatever is stored in `RAM` address 255, and displays that value on 8 bulbs for us to see. You can think of this as our simple output for any programs we might write.

Lastly, we have the `Control Panel`. Its job is to load a program into `RAM` in the first place. It can read and write to any `RAM` address it wants.

How it works is, first you flip the `TAKEOVER` switch, which inside the `CU` basically freezes the computer and resets `PC`. Then, while `TAKEOVER` is on and the `RAM_OUT` button is off, the value on the panel's input switches is put onto the common bus. If `RAM_OUT` were on, then `RAM` would try to drive the bus at the same time as the control panel. That's why we need to make sure it's off before we can safely drive the bus.

From there you can hit `MAR_WRITE`, which stores that as the address you want to work with in `RAM`. To write, you then flip the switches to the value you want and press `RAM_WRITE`. To read, you instead press `RAM_OUT`, which makes the `Control Panel` stop driving the bus, because `RAM` will then drive the bus, and the output bulbs will turn on to that value.

Also, the panel's buttons only actually drive their control wires if `TAKEOVER` is on, using tri-state buffers. This is to ensure that while the CPU is running like normal, the `CU` and the panel don't drive the wires at the same time. This works the other way too: when `TAKEOVER` is on, the `CU` makes sure not to drive those `RAM` control wires, again using tri-state buffers.

Now, about this mysterious box labeled `CU`. What is it exactly? Well, it is the control unit. Think back to most of our previous demos. Turn `O` on, then `W`, flip this control signal and then flip this control signal. The `CU` does all of that flipping for us. It turns the control wires on and off in the right order, at the right times, to make the computer actually work.

But how exactly does the computer work in the first place?

First we load our program into `RAM`. A program is just a set of instructions that tell the CPU what to do, like "move this value here", "add these two numbers", etc. These instructions are coded as numbers.

The control unit then fetches the instruction from `RAM` at the address stored in `PC`, so if `PC` is 0, it fetches the instruction at address 0 and stores it in `IR`. How? It copies `PC` into the `MAR` and flips on `RAM_OUT`, exactly like we have done before. Next it figures out what that instruction means, and executes a bunch of steps to actually do that instruction, then it increments or changes `PC` and repeats that whole cycle.

This is kind of a simplified version of what our CPU will do, but we will dive into it in the following sections. You can think of this cycle as fetch, decode, execute. Fetching gets us the instruction, decoding figures out what it means, and executing actually flips those control wires on and off to accomplish that instruction!

But, how does the `CU` know what wires to flip and when?